

LAPORAN PRAKTIKUM
POSTTEST 5
ALGORITMA PEMROGRAMAN LANJUT



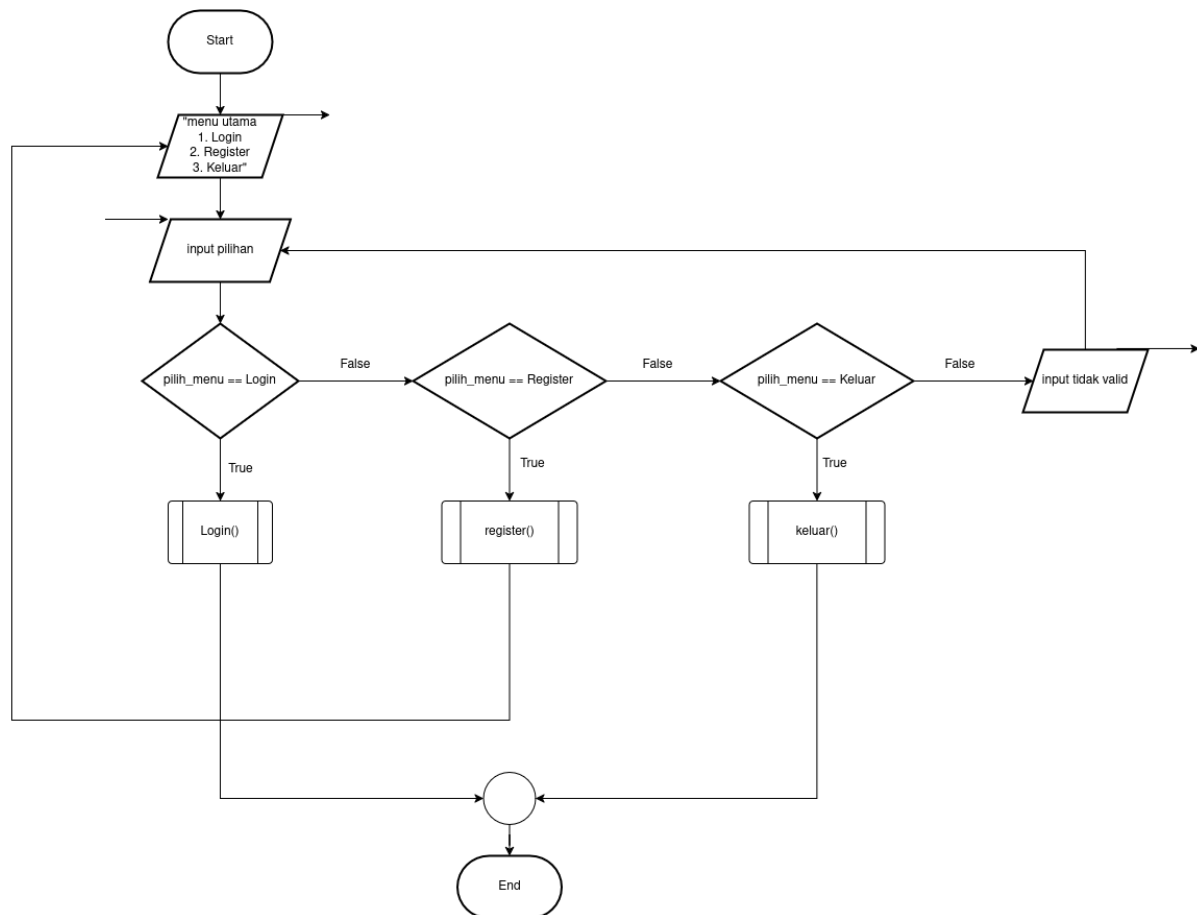
Disusun oleh:
Diftya Azzahra (2509106042)
Kelas (A2'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Flowchart

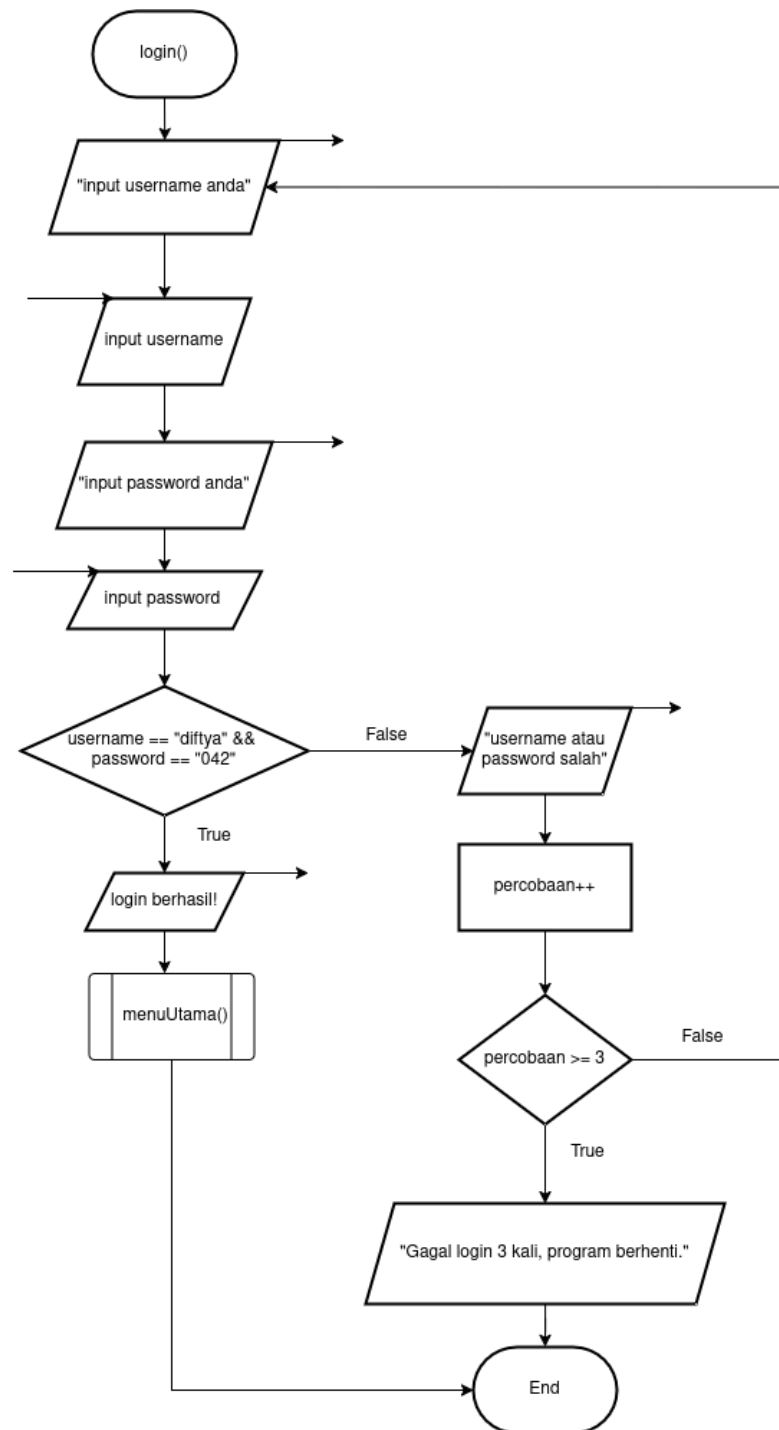
A. Admin

Pada bagian ini, flowchart akan dipisah menjadi bagian admin dan user. Penjelasan akan dimulai dari fitur admin terlebih dahulu



Gambar 1.1 Menu Awal

1. Proses dimulai dari Start, kemudian sistem menampilkan menu awal yang berisi tiga pilihan utama, yaitu login, register, dan keluar. Setelah menu ditampilkan, pengguna diminta untuk memasukkan pilihan sesuai dengan angka yang tersedia.
2. Setelah menginput, sistem melakukan percabangan berdasarkan pilihan pengguna. Jika pengguna memilih login, maka akan masuk ke dalam fungsi loginUser(). Jika pengguna memilih register, maka akan masuk ke dalam fungsi register() dan jika pengguna memilih keluar, maka akan masuk ke dalam fungsi keluar().
3. Jika pengguna menginput selain daripada pilihan yang diberikan, maka input akan dinyatakan tidak valid.

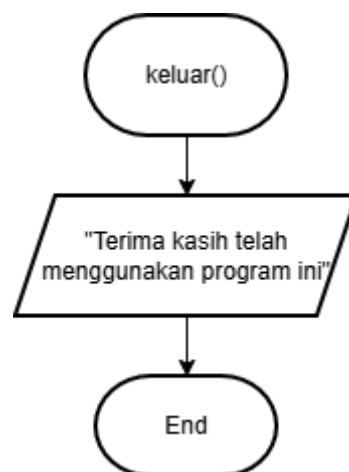


Gambar 1.2 Fungsi Login

4. Proses dimulai dari pemanggilan fungsi login(), di mana sistem akan menampilkan perintah kepada pengguna untuk memasukkan username. Setelah itu, pengguna menginput username serta password untuk lanjut ke tahap berikutnya.
5. Sistem kemudian melakukan pengecekan dengan membandingkan username dan password yang dimasukkan dengan data admin yang telah ditentukan, yaitu username

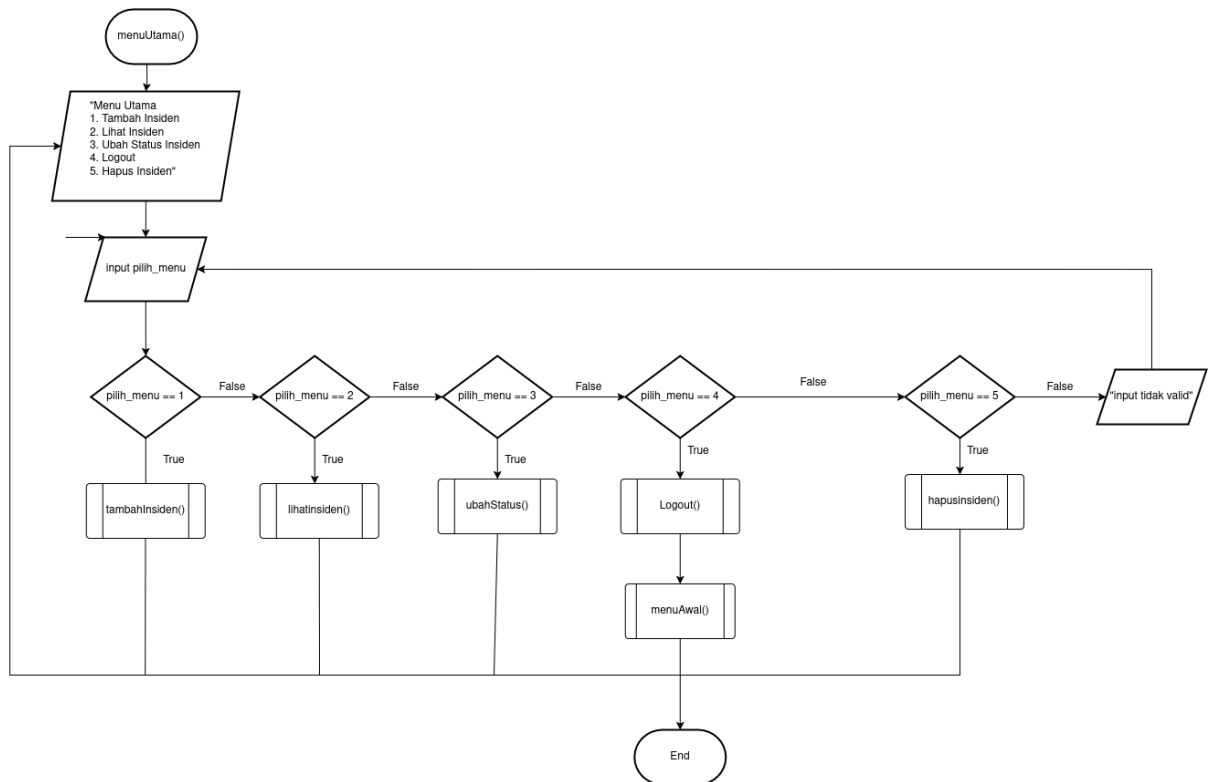
“difitya” dan password “042”. Jika kedua data tersebut sesuai, maka kondisi bernilai *true* dan sistem menampilkan pesan bahwa login berhasil, kemudian pengguna langsung diarahkan ke menu utama melalui pemanggilan fungsi `menuUtama()`.

6. Namun, jika username atau password tidak sesuai, maka kondisi bernilai *false* dan sistem akan menampilkan pesan bahwa username atau password salah. Setelah itu, variabel percobaan akan ditambahkan satu sebagai bentuk pencatatan jumlah kegagalan login yang telah dilakukan oleh pengguna.
7. Sistem kemudian akan melakukan pengecekan terhadap jumlah percobaan login. Jika jumlah percobaan masih kurang dari atau sama dengan tiga, maka pengguna akan dikembalikan ke proses input username dan password untuk mencoba kembali. Namun, jika jumlah percobaan telah melebihi batas yang ditentukan, yaitu lebih dari tiga kali, maka sistem akan menampilkan pesan bahwa login gagal sebanyak tiga kali dan program akan dihentikan.



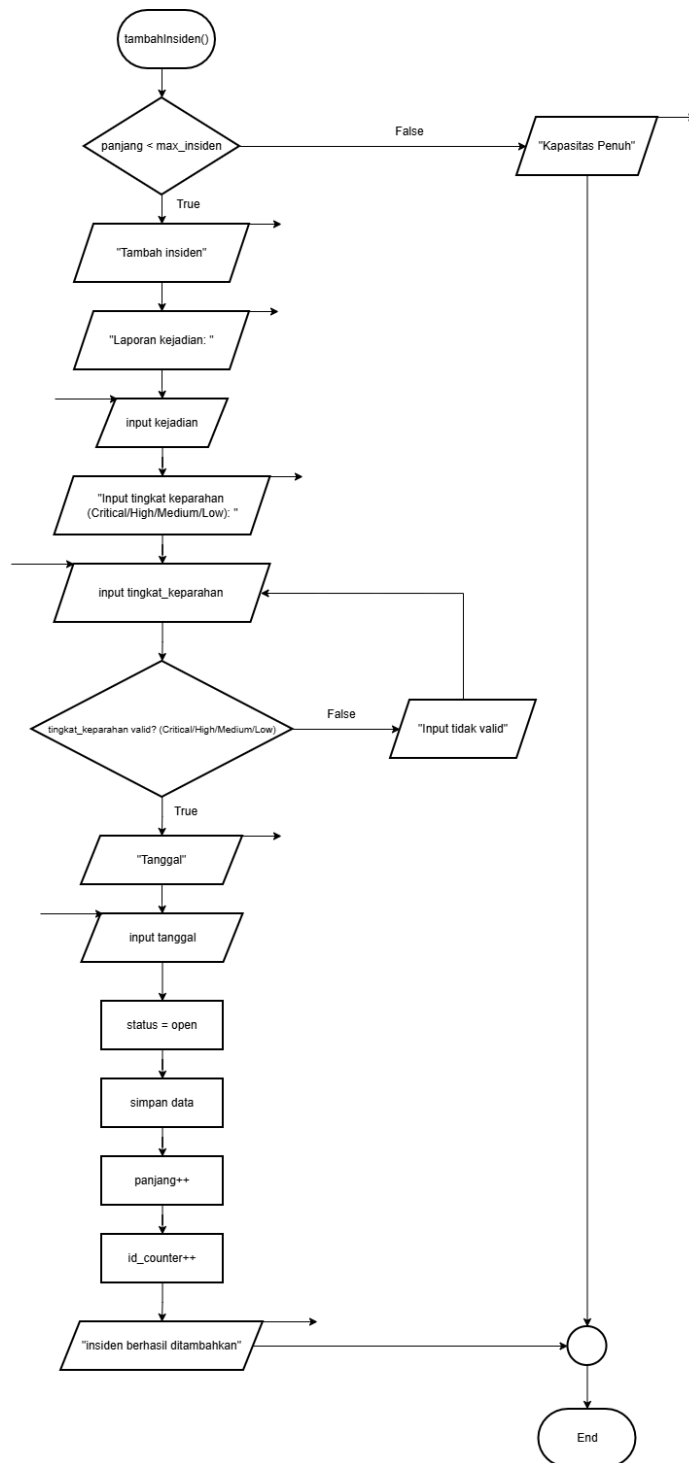
Gambar 1.3 Keluar Program

8. Fungsi ini akan dipanggil ketika pengguna memilih menu ketiga yaitu keluar program. Sistem akan menampilkan output berupa “Terima kasih telah menggunakan program ini” kemudian program pun berhenti.
9. END.



Gambar 1.4 Fungsi Menu Utama

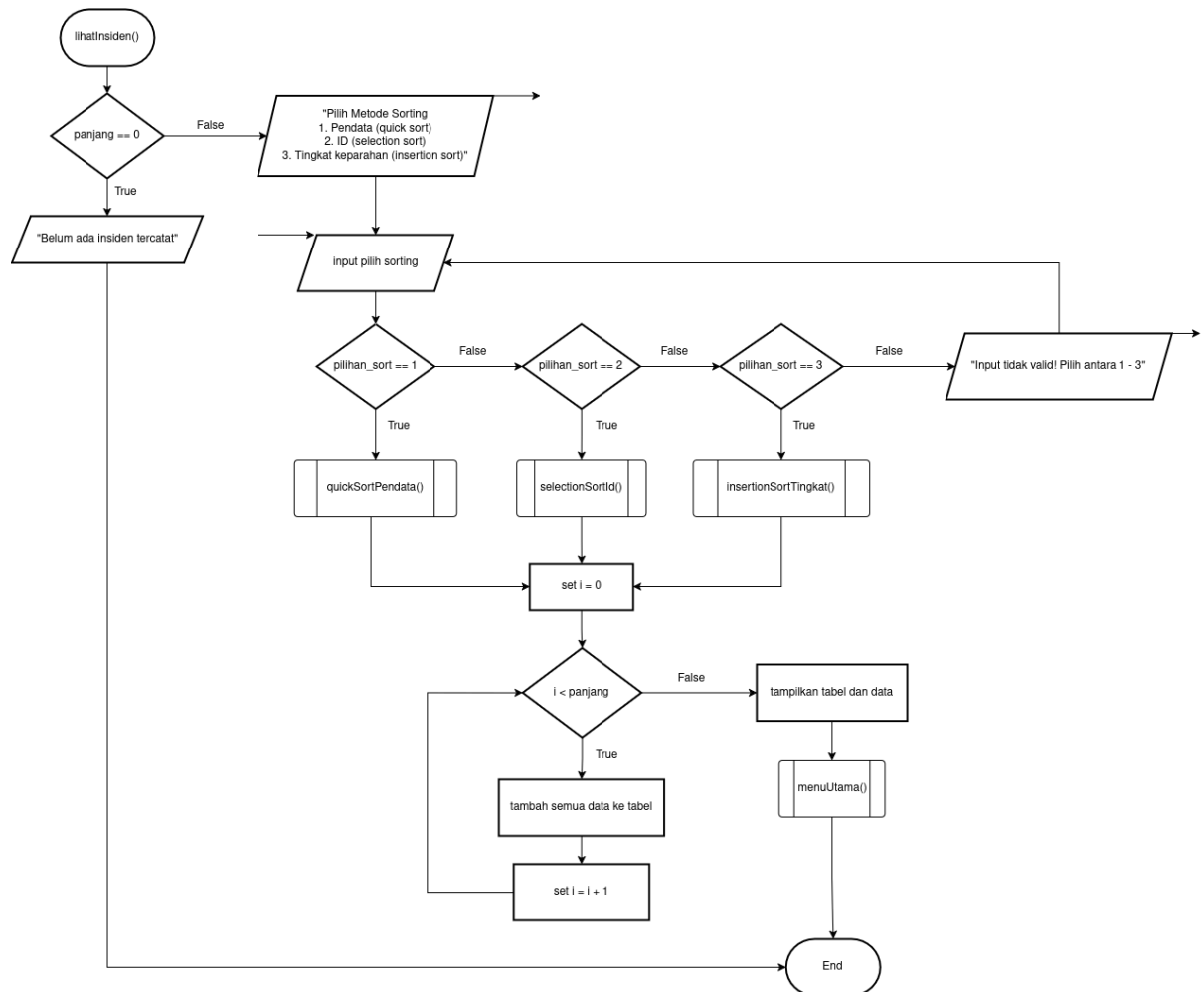
10. Proses dimulai dari pemanggilan fungsi menuUtama(), di mana sistem menampilkan daftar menu utama kepada pengguna yang telah berhasil login sebagai admin. Menu yang ditampilkan terdiri dari lima pilihan, yaitu tambah insiden, lihat insiden, ubah status insiden, logout, dan hapus insiden. Setelah itu, pengguna diminta untuk memasukkan pilihan menu yang diinginkan.
11. Setelah pengguna memasukkan pilihan, sistem akan melakukan pengecekan terhadap nilai input tersebut. Jika pengguna memilih menu dengan nilai 1, maka sistem akan menjalankan fungsi tambahInsiden(). Jika pengguna memilih menu dengan nilai 2, maka sistem akan menjalankan fungsi lihatInsiden(). Jika pengguna memilih menu dengan nilai 3, maka sistem akan menjalankan fungsi ubahStatus(). Jika pengguna memilih menu dengan nilai 4, maka sistem akan menjalankan fungsi logout() dan jika pengguna memilih menu dengan nilai 5, maka sistem akan menjalankan fungsi hapusInsiden().
12. Namun, jika input yang diberikan tidak sesuai dengan pilihan yang tersedia, maka sistem akan menganggapnya sebagai input tidak valid dan tidak menjalankan fungsi apa pun, sehingga sistem akan kembali menunggu input yang benar sesuai dengan implementasi program.



Gambar 1.5 Fungsi Tambah Insiden

13. Proses dimulai dari pemanggilan fungsi `tambahInsiden()`, di mana sistem pertama-tama melakukan pengecekan terhadap kapasitas penyimpanan data insiden dengan membandingkan nilai `panjang` dengan `max_insiden`. Jika kapasitas sudah penuh, maka kondisi bernilai `false` dan sistem akan langsung menampilkan pesan bahwa kapasitas penuh, kemudian proses berakhir tanpa menambahkan data baru.

14. Jika kapasitas masih tersedia, maka kondisi bernilai *true* dan sistem melanjutkan proses dengan menampilkan perintah untuk menambahkan insiden. Selanjutnya pengguna diminta untuk memasukkan laporan kejadian.
15. Setelah laporan kejadian diisi, sistem meminta pengguna untuk memasukkan tingkat keparahan insiden dengan pilihan yang telah ditentukan, yaitu *Critical*, *High*, *Medium*, atau *Low*.
16. Jika tingkat keparahan telah diinput, maka sistem akan melanjutkan dengan meminta input tanggal kejadian.
17. Setelah seluruh data yang dibutuhkan telah lengkap dan valid, sistem akan menetapkan status awal insiden sebagai “*open*”, kemudian seluruh data seperti ID, kejadian, tingkat keparahan, status, tanggal, dan pendata akan disimpan ke dalam struktur data array `dataInsiden()`.
18. Setelah data berhasil disimpan, sistem akan menambahkan nilai panjang sebagai penanda jumlah data insiden yang tersimpan serta menaikkan `id_counter` untuk memastikan setiap insiden memiliki ID yang unik. Terakhir, sistem menampilkan pesan bahwa insiden berhasil ditambahkan.

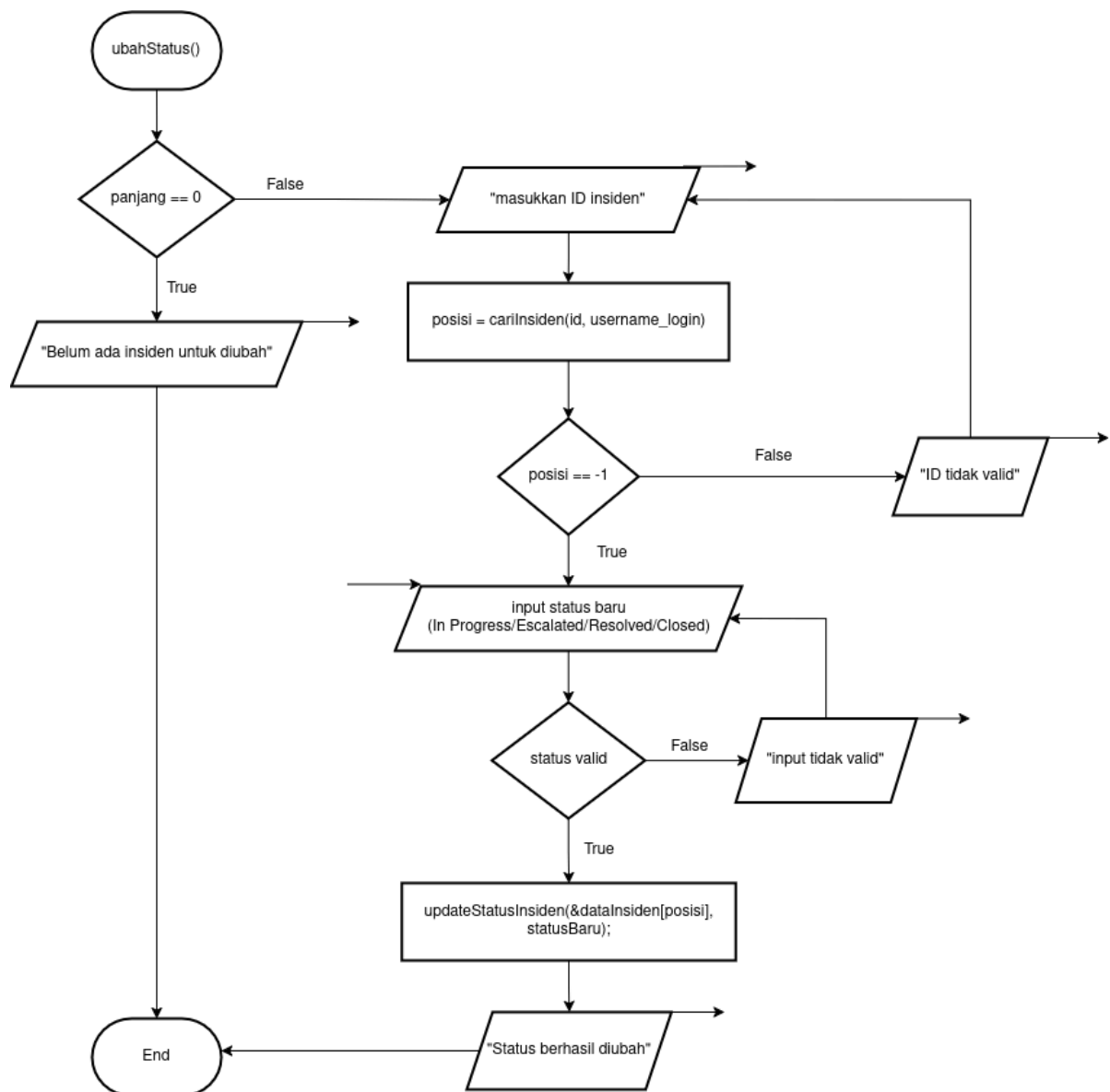


Gambar 1.6 Fungsi Lihat Insiden

19. Proses dimulai dari pemanggilan fungsi `lihatInsiden()` ketika admin memilih menu untuk melihat data insiden. Pada tahap awal, sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan memeriksa kondisi `panjang == 0`. Jika kondisi bernilai `true`, berarti belum ada data insiden yang tersimpan di dalam sistem, sehingga sistem menampilkan pesan “Belum ada insiden tercatat”, lalu proses berakhir.
20. Jika kondisi `panjang == 0` bernilai `false`, maka sistem melanjutkan proses dengan menampilkan pilihan metode sorting kepada admin. Pilihan yang tersedia adalah: (1) Pendata (Quick Sort), (2) ID (Selection Sort), (3) Tingkat Keparahan (Insertion Sort). Setelah pilihan metode sorting ditampilkan, sistem meminta admin untuk melakukan input pilih sorting.
21. Berikutnya, sistem memeriksa nilai input yang dimasukkan. Jika `pilihan_sort == 1`, maka sistem menjalankan fungsi `quickSortPendata()` untuk mengurutkan data berdasarkan nama pendata. Jika `pilihan_sort == 2`, maka sistem menjalankan fungsi

selectionSortId() untuk mengurutkan data dengan ascending berdasarkan ID. Jika pilihan_sort == 3, maka sistem menjalankan fungsi insertionSortTingkat() untuk mengurutkan data berdasarkan tingkat keparahan. Namun, jika input tidak sesuai dengan ketiga pilihan tersebut, maka sistem menampilkan pesan “Input tidak valid! Pilih antara 1 - 3” dan admin diminta untuk melakukan input ulang.

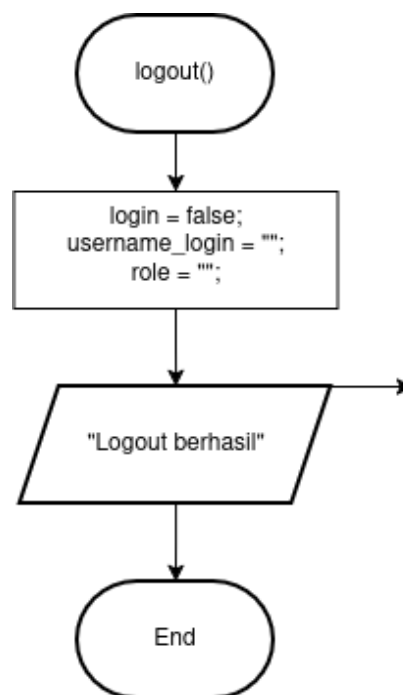
22. Setelah salah satu proses sorting selesai dijalankan, sistem menginisialisasi variabel $i = 0$ untuk memulai penelusuran seluruh data insiden. Selanjutnya, sistem memeriksa kondisi $i < \text{panjang}$. Kondisi ini digunakan untuk menentukan apakah masih ada data insiden yang perlu diproses. Jika kondisi bernilai true, maka sistem akan menambahkan seluruh data ke dalam tabel. Hal ini sesuai dengan hak akses admin yang dapat melihat semua data tanpa perlu melakukan pengecekan apakah data tersebut milik user tertentu atau bukan.
23. Setelah satu data ditambahkan ke tabel, sistem menaikkan nilai i dengan perintah $i = i + 1$, lalu kembali ke pengecekan $i < \text{panjang}$. Proses ini berlangsung berulang sampai seluruh data insiden selesai dimasukkan ke dalam tabel.
24. Jika kondisi $i < \text{panjang}$ bernilai false, berarti seluruh data telah selesai diproses. Setelah itu, sistem menampilkan tabel dan data hasil pengurutan kepada admin. Karena flowchart ini menggambarkan alur dari fungsi yang dipanggil di dalam menu utama, maka setelah tabel ditampilkan alur kembali ke menuUtama(), kemudian proses berakhir pada End.
25. Secara keseluruhan, flowchart lihatInsiden() versi admin menggambarkan bahwa admin dapat melihat seluruh data insiden yang ada di sistem. Sistem hanya perlu memeriksa apakah data tersedia atau tidak, lalu melakukan sorting sesuai pilihan admin, memasukkan semua data ke dalam tabel, dan akhirnya menampilkan tabel tersebut ke layar. Berbeda dengan versi user, pada flowchart admin tidak terdapat proses pengecekan $\text{pendata} == \text{username_login}$, karena admin memang memiliki akses penuh untuk melihat seluruh data insiden.



Gambar 1.7 Fungsi Ubah Status

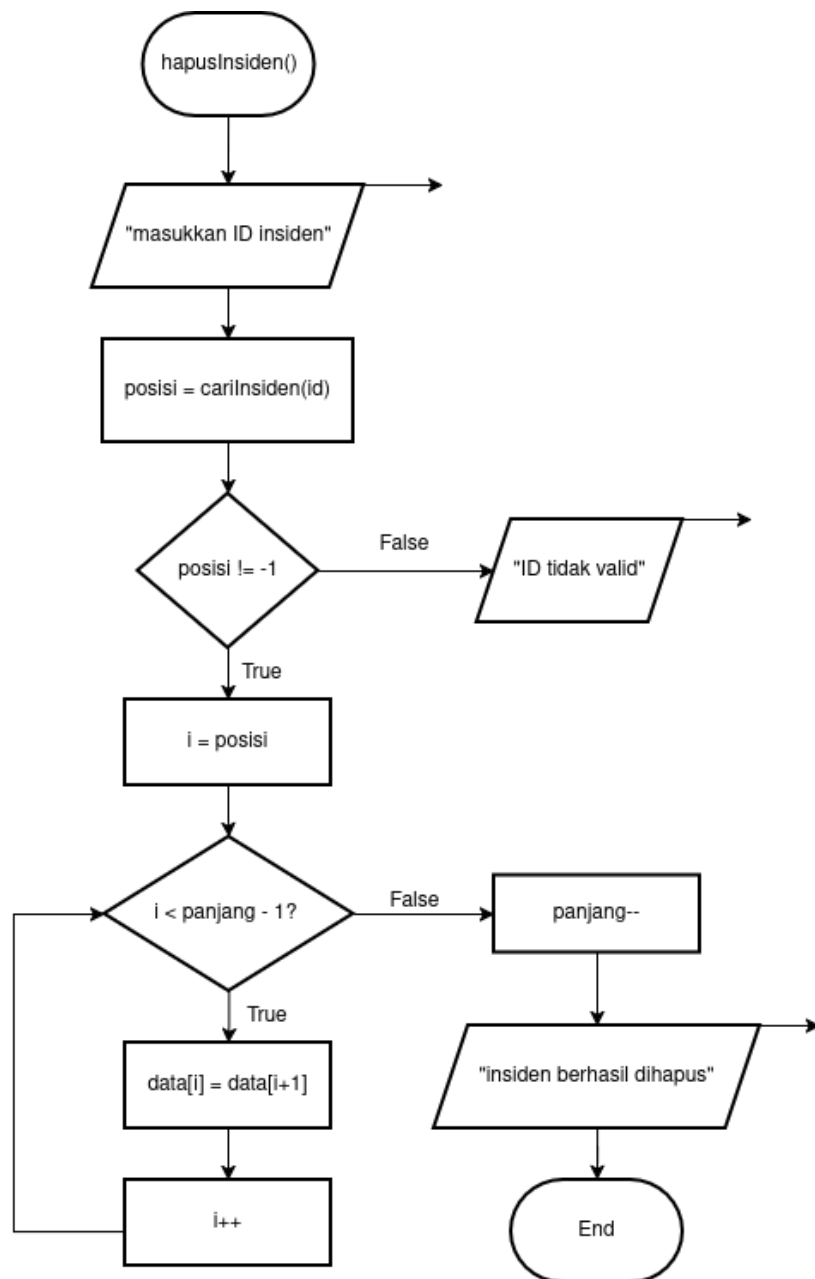
26. Proses dimulai dari pemanggilan menu ubah status insiden pada fungsi menuUtama(), tepatnya pada pilihan menu ke-3.
27. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel panjang. Jika panjang == 0, maka kondisi bernilai true dan sistem menampilkan pesan “Belum ada Insiden untuk diubah”, kemudian proses langsung berakhir tanpa ada perubahan data.
28. Jika terdapat data insiden (panjang != 0), maka kondisi bernilai false dan sistem melanjutkan proses dengan meminta pengguna untuk memasukkan ID insiden.

29. Setelah pengguna memasukkan ID, sistem akan mencari posisi data insiden dalam array menggunakan fungsi cariInsiden, baik untuk admin maupun user sesuai hak aksesnya.
30. Hasil pencarian disimpan dalam variabel posisi. Jika posisi == -1, maka ID tidak ditemukan atau tidak valid, sehingga sistem menampilkan pesan “ID tidak valid!” dan pengguna diminta untuk menginput ulang ID hingga valid.
31. Jika ID valid (posisi != -1), maka sistem melanjutkan dengan meminta pengguna memasukkan status baru untuk insiden tersebut.
32. Pengguna diminta menginput status dengan pilihan yang telah ditentukan, yaitu *In Progress*, *Escalated*, *Resolved*, atau *Closed*.
33. Sistem kemudian melakukan validasi terhadap input status. Jika status yang dimasukkan tidak sesuai dengan pilihan yang tersedia, maka sistem menampilkan pesan “Input tidak valid!” dan pengguna diminta menginput ulang hingga status valid.
34. Jika status yang dimasukkan valid, maka sistem akan melakukan proses pembaruan data dengan mengubah nilai.
35. Setelah proses update berhasil dilakukan, sistem menampilkan pesan “Status berhasil diubah” sebagai konfirmasi bahwa perubahan telah berhasil.
36. Proses kemudian berakhir dan pengguna dikembalikan ke menu utama untuk melanjutkan aktivitas lainnya.



Gambar 1.8 Fungsi Logout

37. Proses dimulai dari pemanggilan fungsi `logout()` pada fungsi `menuUtama()`, tepatnya pada pilihan menu ke-4.
38. Sistem kemudian menjalankan proses penghapusan sesi login dengan mengubah nilai variabel `login` menjadi `false`, yang menandakan bahwa pengguna sudah tidak dalam keadaan login.
39. Selanjutnya, sistem mengosongkan data pengguna yang sedang aktif dengan mengatur nilai `username_login` menjadi string kosong (`""`).
40. Sistem juga mengosongkan informasi peran (role) pengguna dengan mengubah nilai variabel `role` menjadi string kosong (`""`).
41. Setelah seluruh data sesi dihapus, sistem menampilkan pesan “Logout berhasil” sebagai konfirmasi bahwa proses logout telah dilakukan dengan sukses.
42. Proses kemudian berakhir (End), dan pengguna akan dikembalikan ke kondisi awal sistem sebelum login.



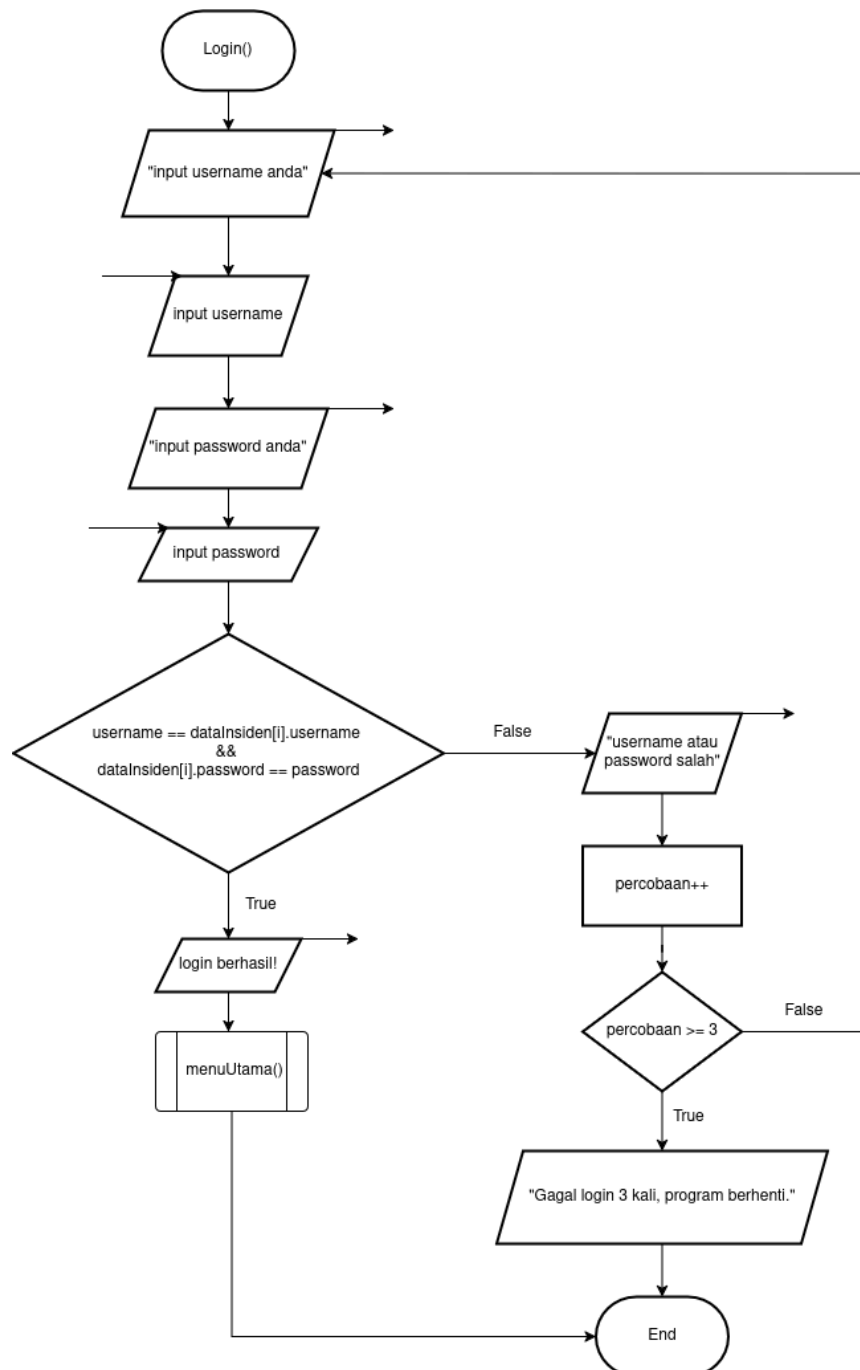
Gambar 1.9 Fungsi HapusInsiden

43. Proses dimulai dari pemanggilan fungsi hapusInsiden() yang terjadi ketika pengguna (admin) memilih menu untuk menghapus data insiden.
44. Sistem kemudian meminta admin untuk memasukkan ID insiden yang ingin dihapus dengan menampilkan pesan “masukkan ID insiden”.
45. Setelah ID dimasukkan, sistem melakukan pencarian data insiden menggunakan fungsi cariInsiden(id) untuk mendapatkan posisi data dalam array.

46. Hasil pencarian disimpan dalam variabel posisi. Jika posisi == -1, maka ID tidak ditemukan atau tidak valid, sehingga sistem menampilkan pesan “ID tidak valid” dan proses berakhir tanpa ada penghapusan data.
47. Jika ID valid (posisi != -1), maka sistem melanjutkan proses dengan menginisialisasi variabel i = posisi sebagai titik awal penghapusan data dalam array.
48. Sistem kemudian melakukan proses pergeseran data menggunakan perulangan dengan kondisi i < panjang - 1.
49. Jika kondisi bernilai true, maka data pada indeks ke-i akan digantikan dengan data pada indeks berikutnya, yaitu data[i] = data[i+1].
50. Setelah proses penyalinan data, nilai indeks i akan ditambahkan satu (i++) untuk melanjutkan pergeseran ke elemen berikutnya.
51. Proses ini akan terus berulang hingga kondisi i < panjang - 1 bernilai false, yang menandakan bahwa seluruh elemen setelah posisi yang dihapus telah digeser ke kiri.
52. Setelah pergeseran selesai, sistem mengurangi jumlah data insiden dengan melakukan panjang-- untuk menghapus elemen terakhir yang sudah tidak digunakan.
53. Sistem kemudian menampilkan pesan “insiden berhasil dihapus” sebagai konfirmasi bahwa proses penghapusan berhasil dilakukan.
54. Proses berakhir (End), dan pengguna dikembalikan ke menu utama.

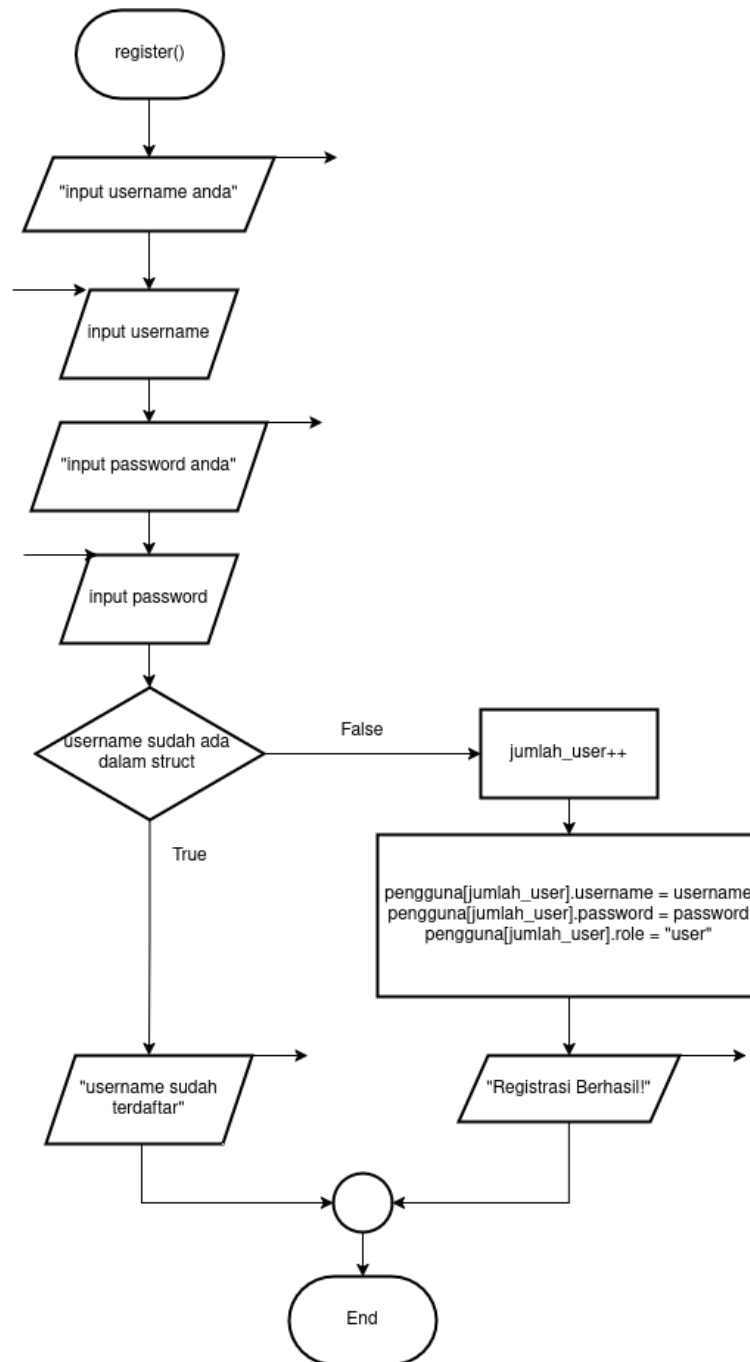
B. User

Kurang lebih seperti admin, hanya saja yang menjadi pembeda hanyalah di beberapa bagian dan beberapa fitur seperti fitur login, register, menu utama, lihat insiden dan ubah status.



Gambar 1.10 Login User

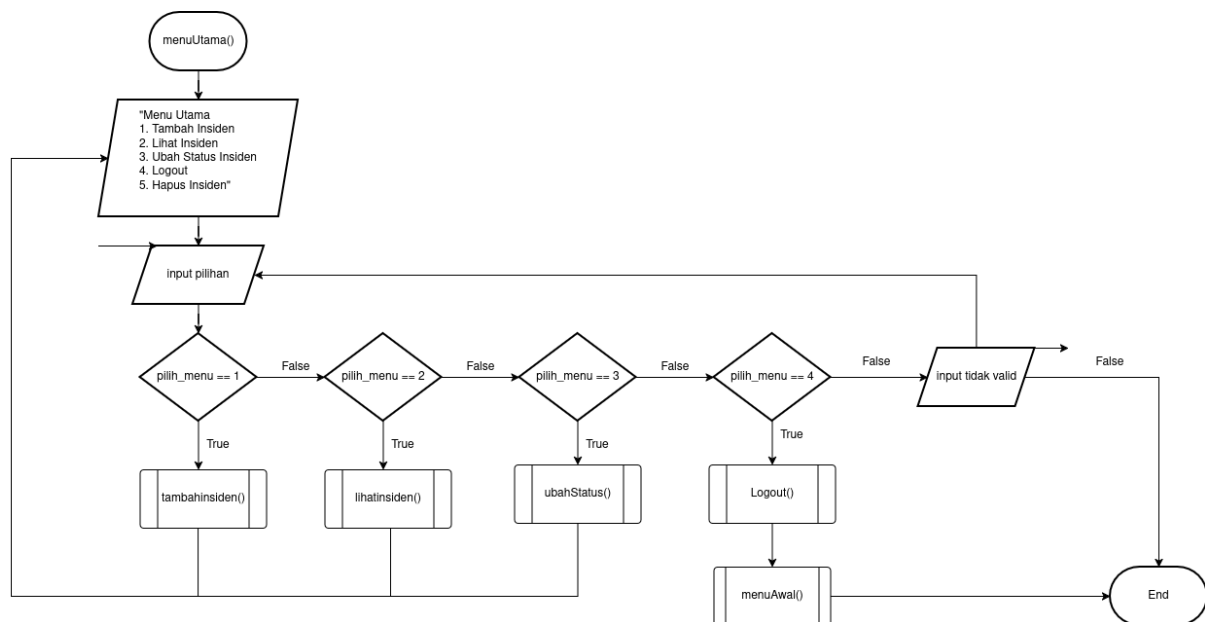
55. Proses dimulai dari pemanggilan fungsi login() ketika user memilih menu login pada sistem.
56. Sistem terlebih dahulu meminta user untuk memasukkan username dengan menampilkan pesan “input username anda”.
57. User kemudian menginput username. Setelah itu, sistem meminta user untuk memasukkan password dengan menampilkan pesan “input password anda”.
58. User menginput password, dan sistem menyimpan nilai tersebut untuk dilakukan proses validasi.
59. Sistem kemudian melakukan pengecekan kecocokan data login dengan membandingkan username dan password yang diinput dengan data yang tersimpan pada sistem.
60. Jika kondisi username dan password sesuai (valid), maka kondisi bernilai true dan sistem menampilkan pesan “login berhasil!”.
61. Setelah login berhasil, proses berakhir (End) dan user dapat mengakses menu utama sesuai role atau hak aksesnya.
62. Jika username atau password tidak sesuai (kondisi false), maka sistem menampilkan pesan “username atau password salah”.
63. Setelah itu, proses akan kembali ke tahap input username dan password, sehingga user dapat mencoba login kembali hingga data yang dimasukkan benar.



Gambar 1.11 Register

64. Proses dimulai dari pemanggilan fungsi register() ketika user memilih menu untuk melakukan pendaftaran akun baru.
65. Sistem terlebih dahulu meminta user untuk memasukkan username dengan menampilkan pesan “input username anda” dan user kemudian menginput username.
66. Setelah itu, sistem meminta user untuk memasukkan password dengan menampilkan pesan “input password anda” dan user pun diminta menginput password.

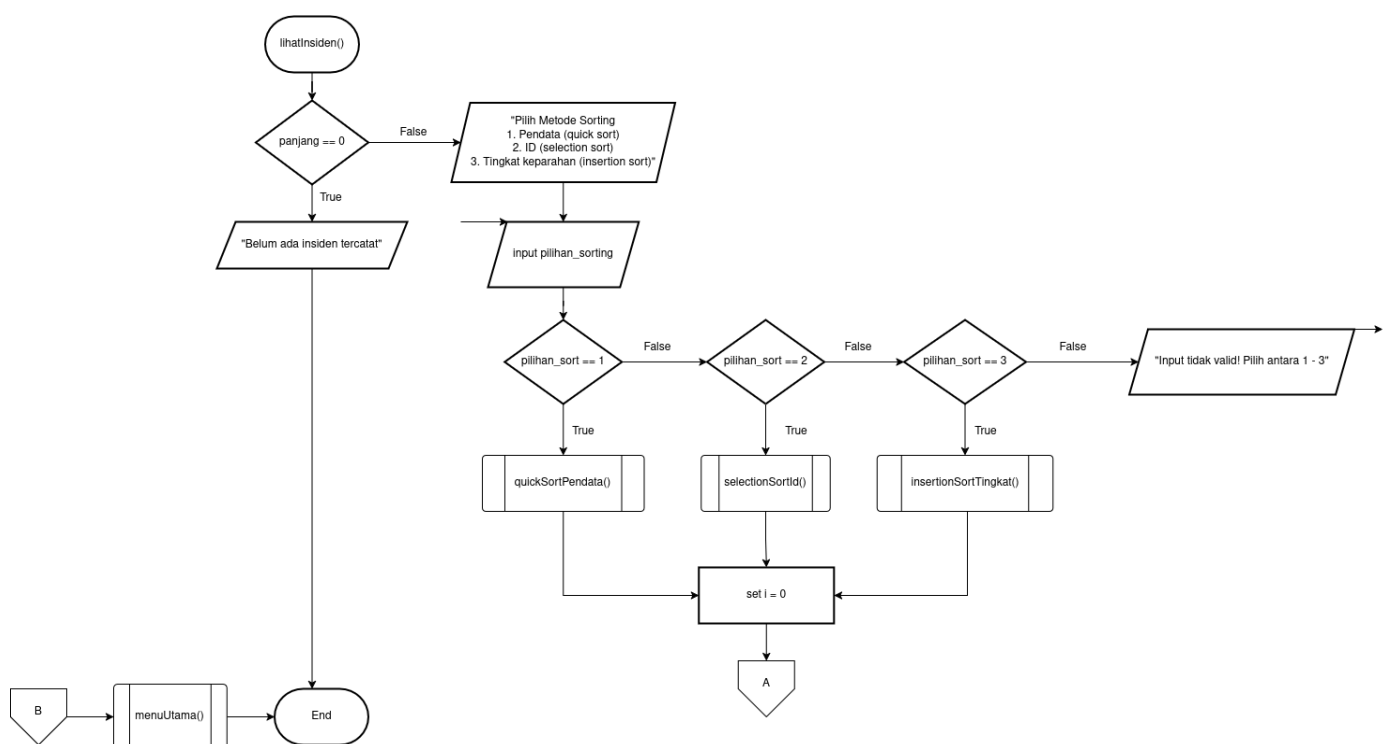
67. Sistem kemudian melakukan pengecekan apakah username yang dimasukkan sudah ada di dalam data (struct pengguna).
68. Jika username sudah terdaftar (kondisi true), maka sistem menampilkan pesan “username sudah terdaftar” dan proses berakhir tanpa menambahkan data baru.
69. Jika username belum terdaftar (kondisi false), maka sistem melanjutkan proses dengan menambahkan jumlah user melalui operasi `jumlah_user++`.
70. Setelah itu, sistem menyimpan data user ke dalam array pengguna dengan mengisi atribut username, password, dan role (sebagai "user").
71. Sistem kemudian menampilkan pesan “Registrasi Berhasil!” sebagai konfirmasi bahwa proses pendaftaran berhasil dilakukan.
72. Proses berakhir (End), dan user dapat kembali ke menu utama untuk melakukan login.



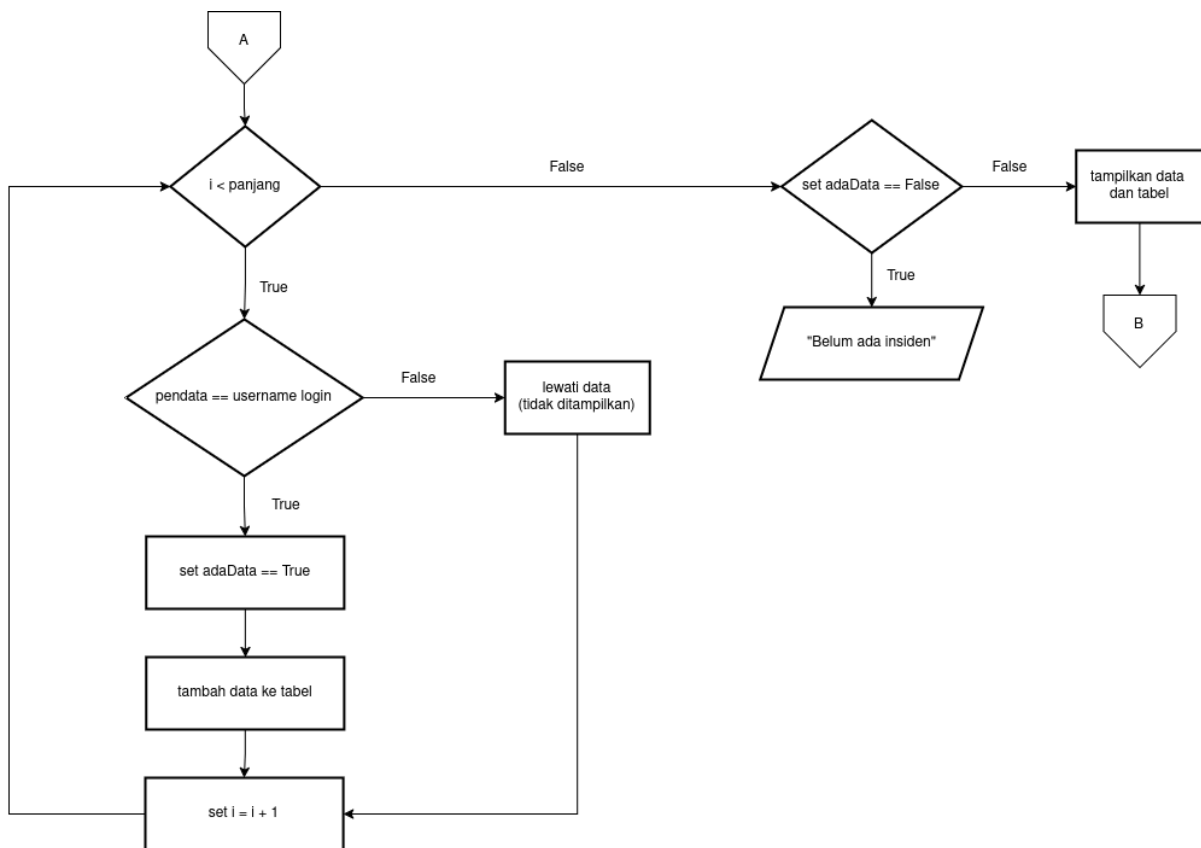
Gambar 1.12 Menu Utama User

73. Proses dimulai dari pemanggilan fungsi `menuUtama()` setelah user berhasil login ke dalam sistem.
74. Sistem kemudian menampilkan daftar menu utama yang dapat diakses oleh user, yaitu: (1) Tambah Insiden, (2) Lihat Insiden, (3) Ubah Status Insiden, (4) Logout.
75. Setelah menu ditampilkan, sistem meminta user untuk memasukkan pilihan menu melalui `input pilih_menu`.
76. Sistem kemudian melakukan pengecekan terhadap nilai input yang dimasukkan oleh user.

77. Jika user memilih menu 1 ($\text{pilih_menu} == 1$), maka sistem akan menjalankan fungsi `tambahinsiden()` untuk menambahkan data insiden baru.
78. Jika user memilih menu 2 ($\text{pilih_menu} == 2$), maka sistem akan menjalankan fungsi `lihatinsiden()` untuk menampilkan daftar insiden yang tersedia.
79. Jika user memilih menu 3 ($\text{pilih_menu} == 3$), maka sistem akan menjalankan fungsi `ubahStatus()` untuk mengubah status insiden.
80. Jika user memilih menu 4 ($\text{pilih_menu} == 4$), maka sistem akan menjalankan fungsi `logout()` untuk keluar dari sistem.
81. Jika input yang dimasukkan tidak sesuai dengan pilihan menu yang tersedia (bukan 1–4), maka sistem menampilkan pesan “input tidak valid”.
82. Kemudian sistem akan kembali ke proses input pilihan menu sampai user memasukkan pilihan yang benar.
83. Proses ini akan terus berulang selama user masih dalam kondisi login, sehingga user dapat terus menggunakan fitur yang tersedia.



Gambar 1.13 Lihat Insiden (User)



Gambar 1.14 Lihat Insiden (User)

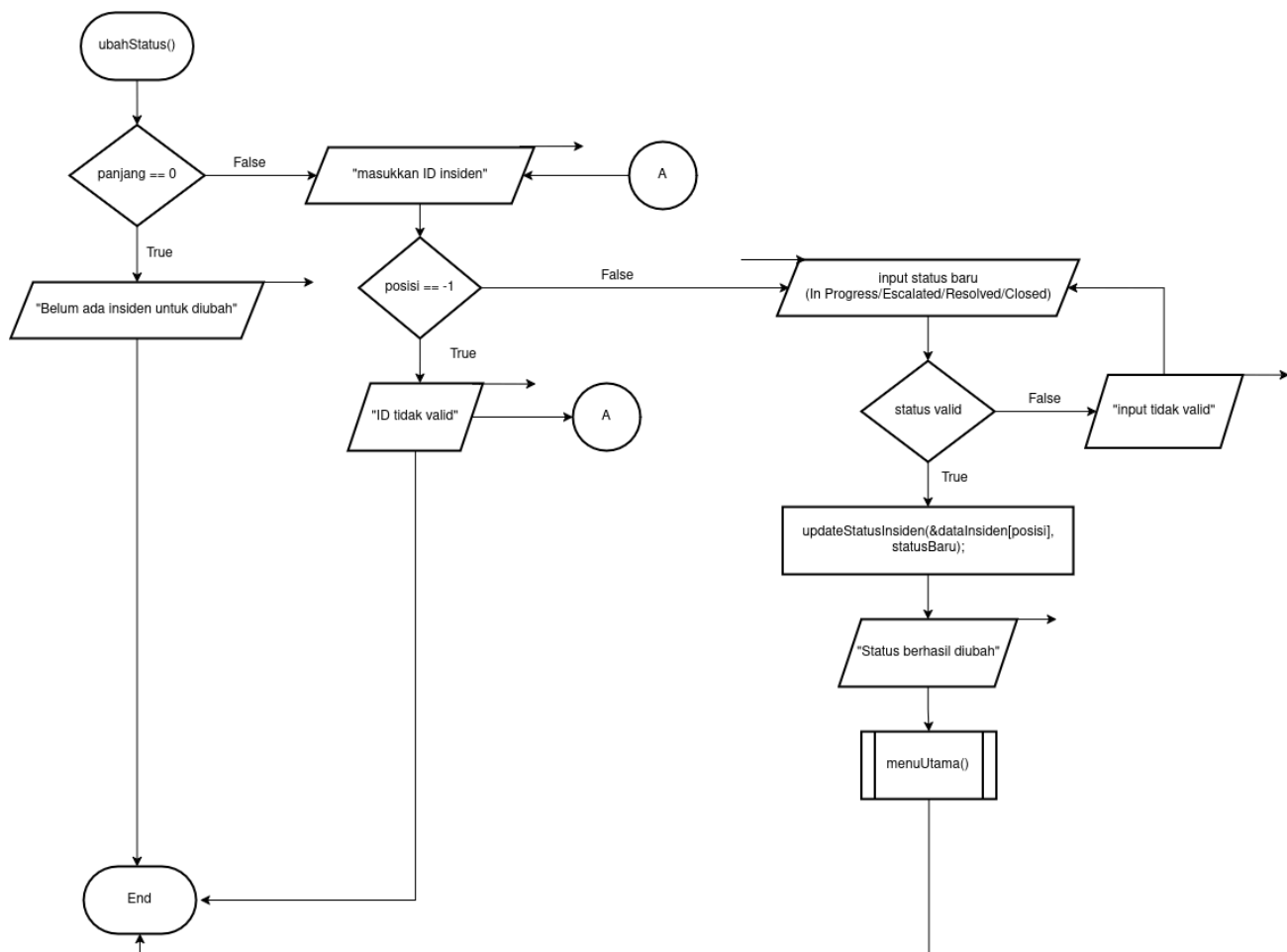
84. Proses dimulai dari pemanggilan fungsi `lihatInsiden()` ketika user memilih menu untuk melihat data insiden. Pada tahap awal, sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan memeriksa kondisi `panjang == 0`. Jika kondisi bernilai `true`, berarti belum ada data insiden yang tersimpan di dalam sistem, sehingga sistem menampilkan pesan “Belum ada insiden tercatat”, lalu proses berakhir.
85. Jika kondisi `panjang == 0` bernilai `false`, maka sistem melanjutkan proses dengan menampilkan pilihan metode sorting kepada user. Pilihan yang tersedia adalah: (1) Pendata (Quick Sort), (2) ID (Selection Sort), (3) Tingkat Keparahan (Insertion Sort). Setelah pilihan ditampilkan, sistem meminta user untuk memasukkan input untuk `pilihan_sorting`.
86. Berikutnya, sistem memeriksa nilai input tersebut melalui beberapa percabangan. Jika `pilihan_sort == 1`, maka sistem menjalankan fungsi `quickSortPendata()` untuk mengurutkan data berdasarkan pendata. Jika `pilihan_sort == 2`, maka sistem menjalankan fungsi `selectionSortId()` untuk mengurutkan data berdasarkan ID. Jika `pilihan_sort == 3`, maka sistem menjalankan fungsi `insertionSortTingkat()` untuk

mengurutkan data berdasarkan tingkat keparahan. Namun, jika input tidak sesuai dengan ketiga pilihan tersebut, maka sistem menampilkan pesan “Input tidak valid! Pilih antara 1 - 3”.

87. Setelah salah satu proses sorting selesai dijalankan, sistem menginisialisasi variabel $i = 0$ untuk memulai proses penelusuran data insiden satu per satu. Selanjutnya, alur berpindah ke bagian lanjutan melalui off-page A. Pada bagian ini, sistem memeriksa kondisi $i < \text{panjang}$ untuk menentukan apakah masih ada data yang perlu diperiksa. Jika kondisi bernilai *true*, maka sistem melanjutkan ke proses pengecekan data berdasarkan user yang sedang login. Jika kondisi bernilai *false*, maka sistem berpindah ke pemeriksaan berikutnya, yaitu kondisi $\text{adaData} == \text{False}$.
88. Selama kondisi $i < \text{panjang}$ bernilai *true*, sistem memeriksa apakah $\text{pendata} == \text{username_login}$. Kondisi ini bertujuan untuk memastikan bahwa data yang sedang diperiksa memang milik user yang sedang login. Jika kondisi bernilai *true*, maka sistem menetapkan $\text{adaData} = \text{True}$, lalu menambahkan data tersebut ke dalam tabel. Setelah itu nilai i dinaikkan dengan perintah $i = i + 1$, kemudian proses kembali ke pengecekan $i < \text{panjang}$ untuk memeriksa data berikutnya.
89. Sebaliknya, jika kondisi $\text{pendata} == \text{username_login}$ bernilai *false*, maka data tersebut tidak ditampilkan karena bukan milik user yang sedang login. Dalam flowchart, langkah ini ditunjukkan dengan proses “lewati data (tidak ditampilkan)”. Setelah data dilewati, nilai i tetap dinaikkan dengan perintah $i = i + 1$, lalu sistem kembali ke pengecekan $i < \text{panjang}$ untuk melanjutkan iterasi ke data berikutnya.
90. Setelah seluruh data selesai diperiksa dan kondisi $i < \text{panjang}$ bernilai *false*, sistem memeriksa kondisi $\text{adaData} == \text{False}$. Kondisi ini digunakan untuk mengetahui apakah selama proses perulangan terdapat data milik user yang berhasil ditemukan atau tidak. Jika kondisi bernilai *true*, berarti tidak ada satupun data insiden yang sesuai dengan username_login , sehingga sistem menampilkan pesan “Belum ada insiden”.
91. Namun, jika kondisi $\text{adaData} == \text{False}$, berarti terdapat data insiden milik user yang berhasil ditemukan dan dimasukkan ke dalam tabel. Dalam kondisi ini, sistem menampilkan data dan tabel ke layar. Setelah itu alur berpindah melalui off-page B lalu kembali ke menu utama, kemudian flowchart berakhir pada End.
92. Secara keseluruhan, flowchart `lihatInsiden()` versi user menggambarkan bahwa sistem hanya akan menampilkan data insiden yang sesuai dengan username user yang sedang login. Meskipun data insiden di sistem ada, jika tidak ada satupun data dengan

pendata yang sama dengan username_login, maka user tetap akan menerima output “Belum ada insiden”. Dengan demikian, proses ini memastikan bahwa user biasa hanya dapat melihat data insiden miliknya sendiri.

93. Setelah seluruh proses selesai, sistem mengakhiri proses (End) dan user dikembalikan ke menu utama.

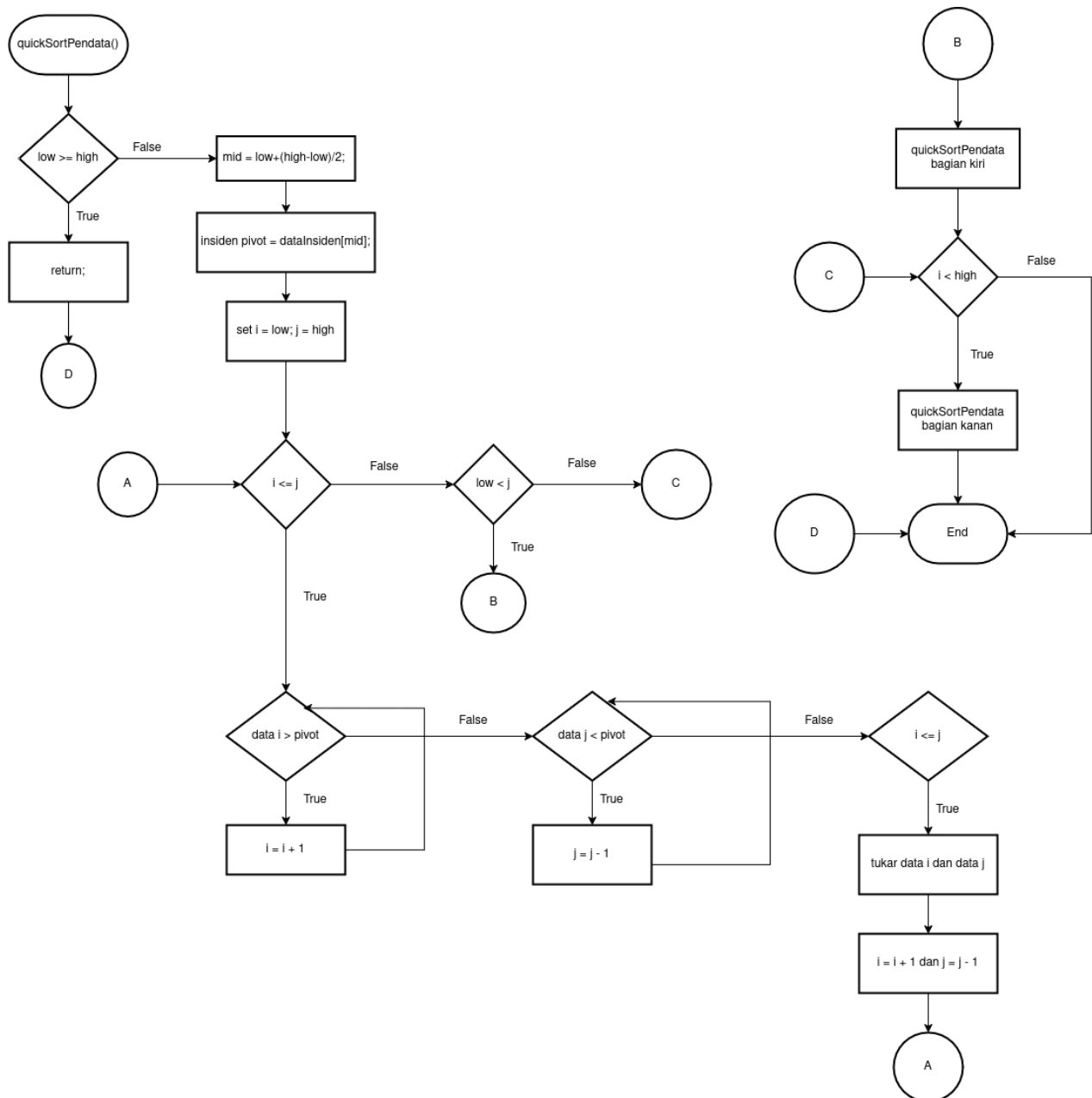


Gambar 1.15 Ubah Status (User)

94. Proses dimulai dari pemanggilan fungsi ubahStatus() ketika user memilih menu untuk mengubah status insiden.
95. Sistem terlebih dahulu melakukan pengecekan terhadap jumlah data insiden dengan melihat nilai variabel panjang. Jika panjang == 0, maka kondisi bernilai true dan sistem menampilkan pesan “Belum ada insiden untuk diubah”, kemudian proses berakhir tanpa melakukan perubahan.

96. Jika terdapat data insiden (panjang $\neq 0$), maka kondisi bernilai false dan sistem melanjutkan proses dengan meminta user untuk memasukkan ID insiden melalui pesan “masukkan ID insiden”.
97. Setelah ID dimasukkan, sistem melakukan pencarian data insiden untuk menentukan posisi data.
98. Jika hasil pencarian menunjukkan posisi $== -1$, maka ID tidak valid sehingga sistem menampilkan pesan “ID tidak valid” dan user diminta untuk menginput ulang ID hingga benar.
99. Jika ID valid (posisi $\neq -1$), maka sistem melanjutkan dengan meminta user untuk memasukkan status baru dengan pilihan In Progress, Escalated, Resolved, Closed.
100. Sistem kemudian melakukan validasi terhadap status yang diinput oleh user.
101. Jika status yang dimasukkan tidak sesuai dengan pilihan yang tersedia, maka sistem menampilkan pesan “input tidak valid” dan user diminta untuk menginput ulang status hingga valid.
102. Jika status yang dimasukkan valid, maka sistem melakukan proses pembaruan dengan mengubah nilai data insiden.
103. Setelah proses update berhasil, sistem menampilkan pesan “Status berhasil diubah” sebagai konfirmasi kepada user.
104. Proses kemudian berakhir (End), dan user dikembalikan ke menu utama untuk melanjutkan aktivitas.

3. Sorting



Gambar 1.16 Fungsi Quick Sort

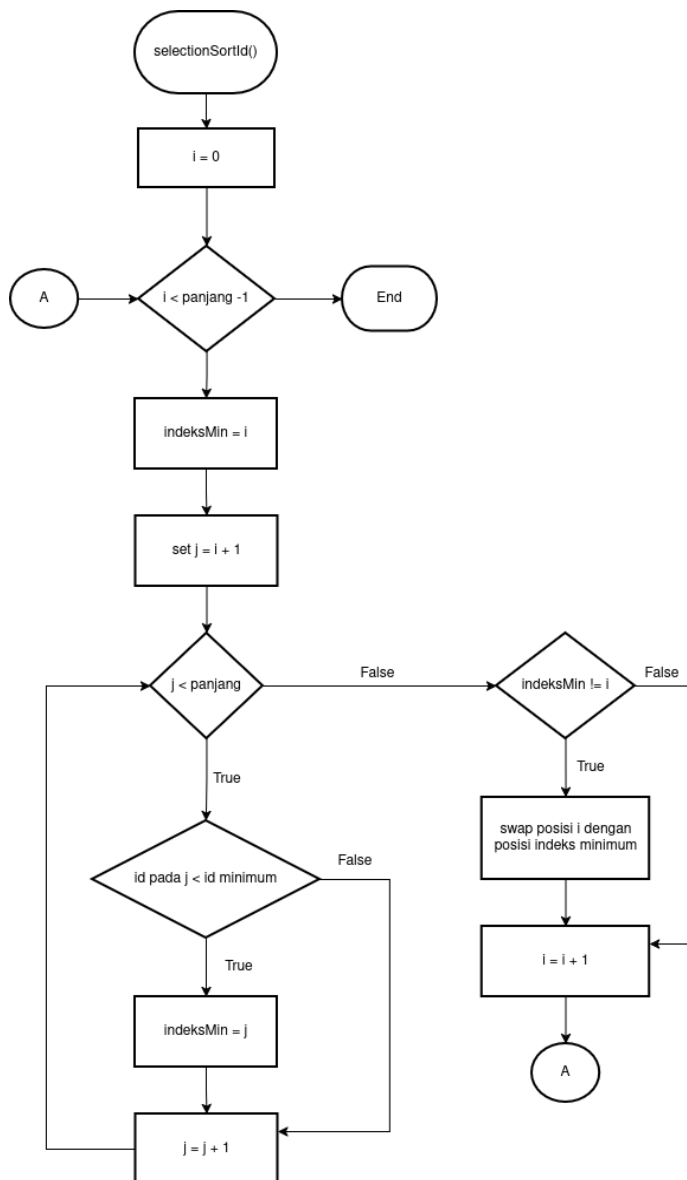
105. Proses dimulai dari pemanggilan fungsi `quickSortPendata(low, high)` ketika user memilih metode pengurutan berdasarkan pendata pada menu lihat insiden. Fungsi ini

digunakan untuk mengurutkan data insiden berdasarkan nama pendata secara descending.

106. Sistem terlebih dahulu melakukan pengecekan kondisi $low \geq high$. Kondisi ini digunakan untuk menentukan apakah bagian data yang sedang diproses masih perlu diurutkan atau tidak. Jika $low \geq high$, maka kondisi bernilai *true* dan fungsi langsung menjalankan return, sehingga proses pada bagian tersebut selesai karena data sudah dianggap terurut atau hanya tersisa satu elemen.
107. Jika $low \geq high$ bernilai *false*, maka sistem melanjutkan proses dengan menghitung nilai tengah menggunakan rumus: **mid** = **low** + (**high** - **low**) / 2. Nilai tengah ini digunakan untuk menentukan **pivot**, yaitu elemen acuan dalam proses partisi. Setelah itu sistem menyimpan nilai pivot dengan perintah: **pivot** = **dataInsiden[mid]**.
108. Selanjutnya sistem menginisialisasi dua variabel indeks, yaitu: $i = low$ dan $j = high$.
109. Variabel i digunakan untuk menelusuri data dari kiri ke kanan, sedangkan j digunakan untuk menelusuri data dari kanan ke kiri.
110. Setelah itu sistem memeriksa kondisi $i \leq j$. Jika kondisi ini bernilai *true*, maka proses partisi dilakukan. Pada tahap ini sistem lebih dulu memeriksa apakah data pada posisi i masih lebih besar dari pivot. Karena pengurutan dilakukan secara descending, selama data pada posisi i masih sesuai dengan aturan urutan, maka nilai i akan terus ditambah dengan perintah $i = i + 1$. Proses ini diulang sampai ditemukan data yang tidak sesuai posisi.
111. Setelah proses dari sisi kiri selesai, sistem memeriksa apakah data pada posisi j lebih kecil dari pivot. Jika kondisi ini bernilai *true*, maka nilai j akan dikurangi dengan perintah $j = j - 1$. Proses ini juga dilakukan berulang sampai ditemukan data yang tidak sesuai posisi dari sisi kanan.
112. Setelah indeks i dan j berhenti pada posisi masing-masing, sistem kembali memeriksa kondisi $i \leq j$. Jika kondisi ini masih bernilai *true*, maka sistem menukar data pada posisi i dan j . Setelah pertukaran dilakukan, sistem memperbarui kedua indeks dengan: $i = i + 1$ dan $j = j - 1$.
113. Kemudian alur kembali ke pengecekan $i \leq j$ untuk melanjutkan proses partisi sampai kedua indeks saling melewati.
114. Jika kondisi $i \leq j$ bernilai *false*, berarti proses partisi pada bagian data tersebut telah selesai. Sistem kemudian memeriksa kondisi $low < j$. Jika kondisi ini bernilai

true, maka masih ada bagian kiri array yang belum terurut, sehingga sistem memanggil kembali fungsi `quickSortPendata(low, j)` untuk mengurutkan bagian kiri.

115. Setelah proses pada bagian kiri selesai, sistem memeriksa kondisi $i < \text{high}$. Jika kondisi ini bernilai true, maka masih ada bagian kanan array yang belum terurut, sehingga sistem memanggil kembali fungsi `quickSortPendata(i, high)` untuk mengurutkan bagian kanan
116. Jika kondisi $\text{low} < j$ bernilai false, maka bagian kiri tidak perlu diproses lagi. Begitu juga jika kondisi $i < \text{high}$ bernilai false, maka bagian kanan juga tidak perlu diproses lagi. Setelah kedua pemeriksaan tersebut selesai, maka proses Quick Sort pada bagian data tersebut berakhir.

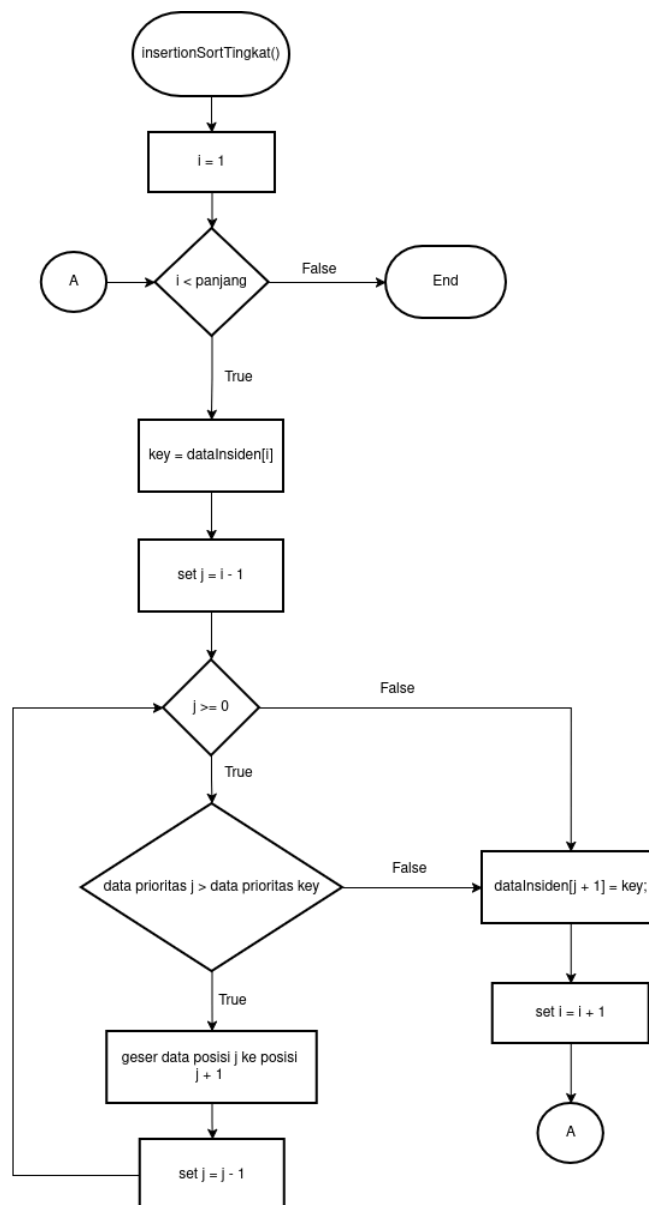


Gambar 1.17 Fungsi Selection Sort

117. Proses dimulai dari pemanggilan fungsi `selectionSortId()` ketika user memilih metode pengurutan berdasarkan ID pada menu lihat insiden. Fungsi ini digunakan untuk mengurutkan data insiden berdasarkan nilai ID secara ascending, yaitu dari nilai terkecil ke nilai terbesar.
118. Pada awal proses, sistem menginisialisasi variabel $i = 0$. Variabel i digunakan sebagai penanda posisi data yang sedang diisi dengan nilai ID terkecil dari bagian data yang belum terurut.
119. Setelah itu sistem memeriksa kondisi $i < \text{panjang} - 1$. Kondisi ini digunakan untuk menentukan apakah masih ada bagian data yang perlu diproses. Jika kondisi bernilai false, berarti seluruh data sudah selesai diurutkan, sehingga proses berakhir. Jika kondisi bernilai true, maka sistem melanjutkan ke langkah berikutnya.
120. Sistem kemudian menetapkan $\text{indeksMin} = i$. Langkah ini berarti data pada posisi i untuk sementara dianggap sebagai data dengan ID paling kecil pada bagian array yang sedang diperiksa. Setelah itu sistem menginisialisasi variabel $j = i + 1$. Variabel j digunakan untuk menelusuri data-data setelah posisi i guna mencari kemungkinan adanya ID yang lebih kecil.
121. Selanjutnya sistem memeriksa kondisi $j < \text{panjang}$. Jika kondisi bernilai true, maka sistem membandingkan ID pada posisi j dengan ID minimum sementara yang berada pada posisi indeksMin . Jika ID pada posisi j lebih kecil dari ID minimum, maka kondisi bernilai true dan sistem memperbarui nilai $\text{indeksMin} = j$. Dengan begitu, posisi minimum berpindah ke indeks yang baru ditemukan.
122. Jika perbandingan menunjukkan bahwa ID pada posisi j tidak lebih kecil dari ID minimum, maka nilai indeksMin tidak berubah. Setelah proses perbandingan selesai, sistem menaikkan nilai j dengan perintah $j = j + 1$, lalu kembali ke pengecekan $j < \text{panjang}$. Langkah ini dilakukan berulang sampai seluruh data di sebelah kanan posisi i selesai diperiksa.
123. Ketika kondisi $j < \text{panjang}$ bernilai false, berarti pencarian nilai minimum pada bagian data tersebut telah selesai. Sistem kemudian memeriksa kondisi $\text{indeksMin} \neq i$. Jika kondisi ini bernilai true, berarti ditemukan data dengan ID yang lebih kecil dari data pada posisi i , sehingga sistem melakukan pertukaran data pada posisi i dengan data pada posisi indeksMin . Tujuan pertukaran ini adalah menempatkan nilai ID terkecil pada posisi yang seharusnya.
124. Jika kondisi $\text{indeksMin} \neq i$ bernilai false, berarti data pada posisi i sudah merupakan data dengan ID terkecil, sehingga pertukaran tidak perlu dilakukan.

Setelah itu, sistem menaikkan nilai i dengan perintah $i = i + 1$, lalu kembali ke pengecekan $i < \text{panjang} - 1$ untuk memproses posisi berikutnya.

125. Proses tersebut terus berulang sampai seluruh data tersusun secara terurut. Dengan demikian, algoritma Selection Sort ID bekerja dengan cara mencari nilai ID terkecil dari bagian data yang belum terurut, lalu menempatkannya pada posisi yang tepat di depan. Setelah itu proses yang sama dilakukan kembali pada sisa data sampai semua data insiden terurut berdasarkan ID dari kecil ke besar.



Gambar 1.18 Fungsi Insertion Sort

126. Proses dimulai dari pemanggilan fungsi `insertionSortTingkat()` ketika user memilih metode pengurutan berdasarkan tingkat keparahan pada menu lihat insiden.

Fungsi ini digunakan untuk mengurutkan data insiden berdasarkan prioritas tingkat keparahan secara ascending, yaitu dari prioritas yang lebih rendah ke prioritas yang lebih tinggi.

127. Pada awal proses, sistem menginisialisasi variabel $i = 1$. Nilai ini dipilih karena elemen pertama dianggap sudah berada pada posisi terurut, sehingga proses penyisipan dimulai dari elemen kedua.
128. Setelah itu sistem memeriksa kondisi $i < \text{panjang}$. Kondisi ini digunakan untuk menentukan apakah masih ada data yang perlu diproses. Jika kondisi bernilai false, berarti seluruh data sudah selesai diurutkan dan proses berakhir. Jika kondisi bernilai true, maka sistem melanjutkan ke langkah berikutnya.
129. Sistem kemudian menyimpan data pada posisi i ke dalam variabel `key` dengan perintah: **`key = dataInsiden[i]`**
130. Variabel `key` berfungsi sebagai data sementara yang akan disisipkan ke posisi yang sesuai dalam bagian data yang sudah terurut. Setelah itu, sistem menginisialisasi variabel $j = i - 1$. Variabel j digunakan untuk menelusuri elemen-elemen di sebelah kiri `key`.
131. Berikutnya sistem memeriksa kondisi $j \geq 0$. Jika kondisi ini bernilai false, berarti tidak ada lagi elemen di sebelah kiri yang perlu dibandingkan, sehingga sistem langsung menempatkan `key` pada posisi $j + 1$. Jika kondisi bernilai true, maka sistem melanjutkan ke proses perbandingan prioritas.
132. Pada tahap ini sistem memeriksa apakah prioritas data pada posisi j lebih besar daripada prioritas `key`. Pemeriksaan ini dilakukan melalui fungsi `prioritasTingkat()`, sehingga yang dibandingkan bukan teks tingkat keparahannya secara langsung, tetapi nilai prioritasnya. Jika kondisi bernilai true, berarti data di sebelah kiri memiliki prioritas yang lebih besar dan harus digeser ke kanan.
133. Ketika kondisi tersebut bernilai true, sistem menggeser data pada posisi j ke posisi $j + 1$. Setelah data digeser, nilai j dikurangi dengan perintah $j = j - 1$. Selanjutnya alur kembali ke pengecekan $j \geq 0$ untuk memeriksa apakah masih ada elemen lain di sebelah kiri yang perlu dibandingkan dengan `key`.
134. Jika kondisi prioritas data j lebih besar dari prioritas `key` bernilai false, berarti posisi yang tepat untuk `key` sudah ditemukan. Dalam kondisi ini, sistem menempatkan `key` ke posisi $j + 1$ dengan perintah: **`dataInsiden[j + 1] = key`**.

135. Setelah key ditempatkan pada posisi yang sesuai, sistem menaikkan nilai i dengan perintah $i = i + 1$, lalu kembali ke pengecekan $i < \text{panjang}$ untuk memproses data berikutnya.
136. Proses tersebut berlangsung berulang sampai seluruh data selesai diperiksa dan disisipkan ke posisi yang benar. Dengan demikian, algoritma Insertion Sort Tingkat Keparahan bekerja dengan cara mengambil satu data sebagai key, lalu membandingkannya dengan data-data sebelumnya. Jika ditemukan data sebelumnya yang memiliki prioritas lebih besar, maka data tersebut digeser ke kanan. Setelah posisi yang tepat ditemukan, key disisipkan pada posisi tersebut. Proses ini diulang sampai seluruh data insiden tersusun berdasarkan tingkat keparahan.

2. Deskripsi Singkat Program

Program ini dapat mempermudah proses perhitungan konversi satuan panjang tanpa perlu menghitung secara manual. Dalam segi pengembang program ini membantu memahami konsep dasar algoritma, flowchart, variabel, percabangan, dan perulangan dalam pemrograman.

3. Source Code

A. Rekursif

```
int hitungid(string user, int index){
    if(index == panjang)
        return 0;

    if(dataInsiden[index].pendata == user)
        return 1 + hitungid(user, index + 1);

    return hitungid(user, index + 1);
}
```

B. Overloading

```
int cariInsiden(int id){
    for(int i = 0; i < panjang; i++)
        if(dataInsiden[i].id == id)
            return i;
    return -1;
}

int cariInsiden(int id, string user){
    for(int i = 0; i < panjang; i++)
```

```

        if(dataInsiden[i].id_user == id && dataInsiden[i].pendata == user)
            return i;
    return -1;
}

```

C. Tambah Insiden

```

void tambahInsiden(string username_login){

    string kejadian, tingkat_keparahan, status, tanggal;

    if(panjang < max_insiden){

        cout << "Tambah Insiden" << endl;
    }
}

```

D. Lihat Insiden

```

void lihatInsiden(string username_login, string role){

    if(panjang == 0)
        cout << "Belum ada insiden" << endl;

    else{

        Table tabel;

        tabel.add_row({"ID", "Kejadian", "Tingkat", "Status", "Tanggal", "Pendata"});

        for(int i = 0; i < panjang; i++)
            if(role == "admin" || dataInsiden[i].pendata == username_login)
                tabel.add_row({
                    to_string(dataInsiden[i].id),
                    dataInsiden[i].kejadian,
                    dataInsiden[i].tingkat,
                    dataInsiden[i].status,
                    dataInsiden[i].tanggal,
                    dataInsiden[i].pendata
                });

        cout << tabel << endl;
    }
}

```

E. Ubah Status Insiden

```

void ubahStatus(string username_login, string role){

    if (panjang == 0) {
        cout << "Belum ada Insiden untuk diubah" << endl;
        return;
    }
}

```

```

    }

    int index, posisi;

    do {
        cout << "Masukkan ID insiden: ";
        cin >> index;
        cin.clear();
        cin.ignore(1000, '\n');

        if(role == "admin")
            posisi = cariInsiden(index);
        else
            posisi = cariInsiden(index, username_login);

        if(posisi == -1)
            cout << "ID tidak valid!" << endl;
    } while (posisi == -1);

    string statusBaru;

    cout << "Status baru (In Progress/Escalated/Resolved/Closed): ";
    getline(cin, statusBaru);

    while(statusBaru != "In Progress" &&
        statusBaru != "Escalated" &&
        statusBaru != "Resolved" &&
        statusBaru != "Closed"){

        cout << "Input tidak valid!" << endl;
        cout << "Masukkan ulang status baru: ";
        getline(cin, statusBaru);
    }

    updateStatusInsiden(&dataInsiden[posisi], statusBaru);

    cout << "Status berhasil diubah" << endl;
}

```

F. Hapus Insiden

```

void hapusInsiden(){

    int index;

    cout << "Masukkan ID insiden: ";
    cin >> index;
    cin.clear();
    cin.ignore(1000, '\n');
}

```

```

int posisi = cariInsiden(index);

if(posisi != -1){

    for(int i = posisi; i < panjang-1; i++)
        dataInsiden[i] = dataInsiden[i+1];

    panjang--;

    cout << "Insiden berhasil dihapus" << endl;
}
else
    cout << "ID tidak valid" << endl;
}

```

G. Menu Utama

```

void menuUtama(string &username_login, string &role){

    int pilih_menu;

    while(login){

        if(role == "user"){
            cout << "\n=== MENU UTAMA ===" << endl;
            cout << "Login sebagai: " << username_login << " (" << role << ")" <<
endl;

            cout << "1. Tambah Insiden" << endl;
            cout << "2. Lihat Insiden" << endl;
            cout << "3. Ubah Status Insiden" << endl;
            cout << "4. Logout" << endl;
        }
        else{
            cout << "\n=== MENU UTAMA ===" << endl;
            cout << "Login sebagai: " << username_login << " (" << role << ")" <<
endl;

```

H. Main

```

int main(){

    int pilihan;
    string username, password;
    string username_login = "";
    string role = "";

    int percobaan_login = 0;

    while(true){

```

```

if(!login){

    cout << "\nSISTEM MANAJEMEN INSIDEN (SIEM)" << endl;
    cout << "1. Login" << endl;
    cout << "2. Register" << endl;
    cout << "3. Keluar" << endl;

    cout << "Pilih menu: ";
    cin >> pilihan;

    if(cin.fail()){
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "Input tidak valid" << endl;
        continue;
    }

    if(pilihan < 1 || pilihan > 3){
        cout << "Input tidak valid" << endl;
        continue;
    }

    switch(pilihan){

    case 1:

        while(percobaan_login < 3 && !login){
            cin.ignore(1000, '\n');

            do {
                cout << "Input Username Anda: ";
                getline(cin, username);

                if (username == "") {
                    cout << "Username tidak boleh kosong!" << endl;
                }
            }while(username == "");

            do {
                cout << "Input Password Anda: ";
                getline(cin, password);

                if (password == "")
                    cout << "Password tidak boleh kosong!" << endl;
            }while (password == "");

            if(cin.fail()){
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "Input tidak valid" << endl;
                continue;
            }
        }
    }
}

```

```

    }

    if(loginUser(username,password,role)){

        login = true;
        username_login = username;
        percobaan_login = 0;

        cout << "Login Berhasil!" << endl;

        menuUtama(username_login,role);
        break;
    }
    else{

        percobaan_login++;

        cout << "Username atau password salah" << endl;

        if(percobaan_login >= 3){
            cout << "Gagal login 3 kali, Program berhenti." <<
endl;
            return 0;
        }
    }
}

break;

case 2:

    if(jumlah_user < max_user){

        cin.ignore(1000, '\n'); // buang newline sisa sebelumnya

        cout << "input username anda: ";
        getline(cin, username);

        cout << "input password anda : ";
        getline(cin, password);

        if(registerUser(username,password))
            cout << "Registrasi Berhasil!" << endl;
        else
            cout << "Username sudah terdaftar" << endl;
    }
    else
        cout << "Kapasitas user penuh" << endl;

```

J. Helper

```
bool loginUser(string username, string password, string &role){
    if(username == "diftya" && password == "042"){
        role = "admin";
        return true;
    }

    for(int i = 0; i < jumlah_user; i++){
        if(pengguna[i].username == username && pengguna[i].password == password){
            role = "user";
            return true;
        }
    }

    return false;
}

bool registerUser(string username, string password){
    if(username == "diftya")
        return false;

    for(int i = 0; i < jumlah_user; i++){
        if(pengguna[i].username == username)
            return false;
    }

    pengguna[jumlah_user].username = username;
    pengguna[jumlah_user].password = password;
    pengguna[jumlah_user].role = "user";

    jumlah_user++;
    return true;
}
```

K. Dereference (Parameter Fungsi)

```
void updateStatusInsiden(insiden *data, string statusBaru){
    data->status = statusBaru;
}
```

L. Address-of Operator (Parameter Fungsi)

```
updateStatusInsiden(&dataInsiden[posisi], statusBaru);
```

M. Pointer pada struct

```
insiden *data
```

N. Quick Sort (Descending Nama Pendata Insiden)

```
bool lebihKecilPendata(insiden a, insiden b){
    return a.pendata > b.pendata;
}
```

```

}
bool lebihBesarPendata(insiden a, insiden b){
    return a.pendata < b.pendata;
}
void quickSortPendata(int low, int high){
    if(low >= high)
        return;

    int mid = low + (high - low) / 2;
    insiden pivot = dataInsiden[mid];

    int i = low;
    int j = high;

    while(i <= j){
        while(i <= high && lebihKecilPendata(dataInsiden[i], pivot)){
            i++;
        }

        while(j >= low && lebihBesarPendata(dataInsiden[j], pivot)){
            j--;
        }

        if(i <= j){
            swap(dataInsiden[i], dataInsiden[j]);
            i++;
            j--;
        }
    }

    if(low < j)
        quickSortPendata(low, j);

    if(i < high)
        quickSortPendata(i, high);
}

```

O. Selection Sort (Ascending ID)

```

void selectionSortId(){
    for(int i = 0; i < panjang - 1; i++){
        int indeksMin = i;

        for(int j = i + 1; j < panjang; j++){
            if(dataInsiden[j].id < dataInsiden[indeksMin].id){
                indeksMin = j;
            }
        }
    }
}

```

```

        if(indeksMin != i){
            swap(dataInsiden[i], dataInsiden[indeksMin]);
        }
    }
}

```

P. Insertion Sort (Ascending Tingkat Keparahan)

```

void insertionSortTingkat(){
    for(int i = 1; i < panjang; i++){
        insiden key = dataInsiden[i];
        int j = i - 1;

        while(j >= 0 &&
            prioritasTingkat(dataInsiden[j].tingkat) >
            prioritasTingkat(key.tingkat)){
            dataInsiden[j + 1] = dataInsiden[j];
            j--;
        }

        dataInsiden[j + 1] = key;
    }
}

```

4. Hasil Output

```

SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: diftya
Input Password Anda: 042
Login Berhasil!

```

Gambar 4.1 Output program login berhasil

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: yaya
Input Password Anda: biskuit yaya
Username atau password salah

Input Username Anda: █
```

Gambar 4.2 Output program ketika login salah

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 1
Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah

Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah

Input Username Anda: makan
Input Password Anda: nasi
Username atau password salah
Gagal login 3 kali, Program berhenti.
```

Gambar 4.2 Output Program ketika login salah (percobaan ≥ 3)

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 2
input username anda: yaya
input password anda : biskuit yaya
Registrasi Berhasil!
```

Gambar 4.3 Output Register Berhasil

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 2
input username anda: yaya
input password anda : rengginang
Username sudah terdaftar
```

Gambar 4.4 Output Register Gagal

```
SISTEM MANAJEMEN INSIDEN (SIEM)
1. Login
2. Register
3. Keluar
Pilih menu: 3
Terima kasih telah menggunakan program ini
```

Gambar 4.5 Output Menu Keluar Program

```
=== MENU UTAMA ===
Login sebagai: diftya (admin)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
5. Hapus Insiden
Pilih menu: █
```

Gambar 4.6 Output Menu Utama Admin

```
=== MENU UTAMA ===
Login sebagai: yaya (user)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
Pilih menu: 
```

Gambar 4.7 Output Menu utama user

```
Pilih menu: 1
Tambah Insiden
Laporan kejadian: Kebocoran data
Input tingkat keparahan (Critical/High/Medium/Low): Critical
Tanggal: 15 Maret 2026
Insiden berhasil ditambahkan!
```

Gambar 4.8 Output Tambah Insiden

```
Pilih menu: 2
+---+-----+-----+-----+-----+
| ID | Kejadian      | Tingkat | Status | Tanggal      | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran data | Critical | open   | 15 Maret 2026 | yaya    |
+---+-----+-----+-----+-----+
```

Gambar 4.9 Output Lihat Insiden

```
Pilih menu: 2
+---+-----+-----+-----+-----+
| ID | Kejadian      | Tingkat | Status   | Tanggal      | Pendata |
+---+-----+-----+-----+-----+
| 1  | Kebocoran data | Critical | Escalated | 15 Maret 2026 | yaya    |
+---+-----+-----+-----+-----+
```

Gambar 4.10 Output Ubah Status Insiden

```
=== MENU UTAMA ===
Login sebagai: yaya (user)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
Pilih menu: 4
Logout berhasil!
```

Gambar 4.11 Output Logout

```
=== MENU UTAMA ===
Login sebagai: diftya (admin)
1. Tambah Insiden
2. Lihat Insiden
3. Ubah Status Insiden
4. Logout
5. Hapus Insiden
Pilih menu: 5
Masukkan ID insiden: 1
Insiden berhasil dihapus
```

Gambar 4.12 Output Hapus Insiden (Fitur Admin)

```
=== Pilih Metode Sorting ===
1. Pendata (Quick Sort)
2. ID (Selection Sort)
3. Tingkat Keparahan (Insertion Sort)
Pilih sorting: 1
+---+-----+-----+-----+-----+
| ID | Kejadian          | Tingkat | Status | Tanggal       | Pendata |
+---+-----+-----+-----+-----+
| 2  | Peretasan Server  | High    | open   | 4 April 2026  | yaya    |
+---+-----+-----+-----+-----+
| 3  | Hilangnya beberapa data | Medium  | open   | 13 April 2026 | fitri   |
+---+-----+-----+-----+-----+
| 1  | Kebocoran Data    | Critical | open   | 7 April 2026  | diftya  |
+---+-----+-----+-----+-----+
```

Gambar 4.13 Output Sorting pada Nama Pendata (Descending Quick Sort)

```
=== Pilih Metode Sorting ===
```

1. Pendata (Quick Sort)
2. ID (Selection Sort)
3. Tingkat Keparahan (Insertion Sort)

```
Pilih sorting: 2
```

```
+-----+-----+-----+-----+-----+-----+
| ID | Kejadian          | Tingkat | Status | Tanggal      | Pendata |
+-----+-----+-----+-----+-----+-----+
| 1  | Kebocoran Data     | Critical | open  | 7 April 2026 | diftya  |
+-----+-----+-----+-----+-----+-----+
| 2  | Peretasan Server   | High    | open  | 4 April 2026 | yaya    |
+-----+-----+-----+-----+-----+-----+
| 3  | Hilangnya beberapa data | Medium  | open  | 13 April 2026 | fitri   |
+-----+-----+-----+-----+-----+-----+
```

Gambar 4.14 Output Sorting pada ID (Ascending Selection Sort)

```
=== Pilih Metode Sorting ===
```

1. Pendata (Quick Sort)
2. ID (Selection Sort)
3. Tingkat Keparahan (Insertion Sort)

```
Pilih sorting: 3
```

```
+-----+-----+-----+-----+-----+-----+
| ID | Kejadian          | Tingkat | Status | Tanggal      | Pendata |
+-----+-----+-----+-----+-----+-----+
| 3  | Hilangnya beberapa data | Medium  | open  | 13 April 2026 | fitri   |
+-----+-----+-----+-----+-----+-----+
| 2  | Peretasan Server   | High    | open  | 4 April 2026 | yaya    |
+-----+-----+-----+-----+-----+-----+
| 1  | Kebocoran Data     | Critical | open  | 7 April 2026 | diftya  |
+-----+-----+-----+-----+-----+-----+
```

Gambar 4.15 Output Sorting pada Tingkat Keparahan (Ascending Insertion Sort)

5. Langkah-langkah GIT

5.1 GIT Add

```
~/Documents/praktikum-apl/posttest main*
• > git add post-test-apl-5/
```

Gambar 5.1 Git Add

Dilakukan penambahan *project* ke github yaitu dengan git add.

5.2 GIT Commit

```
~/Documents/praktikum-apl/posttest main*  
● > git commit -m "Finish Posttest-5"  
[main 7131687] Finish Posttest-5  
2 files changed, 598 insertions(+)  
create mode 100644 posttest/post-test-apl-5/2509106042-Diftya_Azzahra-PT-5.cpp  
create mode 100755 posttest/post-test-apl-5/run
```

Gambar 5.2 Git Commit

Dilakukan commit ke github yaitu dengan *message* berupa finish posttest-5.

5.3 GIT Push

```
~/Documents/praktikum-apl/posttest main* ↑  
● > git push -u origin main  
Enumerating objects: 8, done.  
Counting objects: 100% (8/8), done.  
Delta compression using up to 4 threads  
Compressing objects: 100% (5/5), done.  
Writing objects: 100% (6/6), 102.45 KiB | 7.88 MiB/s, done.  
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/HikaruYui/praktikum-apl.git  
4dde283..06afffc main -> main  
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3 Git Push

Posttest pun berhasil terpush ke github